Math & Bit Manipulation

> → **Key Idea**: Use mathematical properties and bit operations for efficient computation

> • **When to use**: Number theory, prime numbers, bit operations, mathematical optimization

> ⊕ **Complexity**: Varies by algorithm, bit operations typically O(1) or O(log n)

> ⚠ **Common Pitfall**: Integer overflow, incorrect bit manipulation, or not using mathematical shortcuts

Overview

Math and bit manipulation problems leverage number properties and binary representation for efficient solutions. Key topics include prime numbers, GCD/LCM, modular arithmetic, and bitwise operations.

Visual Example

Bit Manipulation Operations

```
Number: 12 = 1100 in binary

Operations:
  AND (&):  1100 & 1010 = 1000 (8)   - Keeps bits where both are 1
  OR  (|):  1100 | 1010 = 1110 (14)  - Sets bit if either is 1
  XOR (^):  1100 ^ 1010 = 0110 (6)   - Sets bit if exactly one is 1
  NOT (~):  ~1100 = ...0011 (-13)    - Flips all bits
  LEFT:     1100 << 2 = 110000 (48)  - Multiply by 2^n
  RIGHT:    1100 >> 2 = 11 (3)       - Divide by 2^n
```

Common Bit Tricks

```
Check if n is power of 2:     n & (n-1) == 0
  8 = 1000,  7 = 0111  →  1000 & 0111 = 0000 ✓

Get lowest set bit:           n & (-n)
  12 = 1100,  -12 = 0100  →  1100 & 0100 = 0100 (4)

Count set bits (Brian Kernighan):
  n = 1100  →  n & (n-1) = 1000  →  n & (n-1) = 0000
  Count = 2
```

```
Clear lowest set bit:          n & (n-1)
  12 = 1100  →  12 & 11 = 1100 & 1011 = 1000 (8)
```

Sieve of Eratosthenes (Prime Numbers)

```
Find primes up to 30:

Initial:  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20...30
Cross 2:  2  3  X  5  X  7  X  9  X 11  X 13  X 15  X 17  X 19  X...
Cross 3:  2  3  X  5  X  7  X  X  X 11  X 13  X  X  X 17  X 19  X...
Cross 5:  2  3  X  5  X  7  X  X  X 11  X 13  X  X  X 17  X 19  X...

Primes: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29
```

GCD with Euclidean Algorithm

```
GCD(48, 18):
  48 = 18 × 2 + 12    →  GCD(18, 12)
  18 = 12 × 1 + 6     →  GCD(12, 6)
  12 = 6 × 2 + 0      →  GCD(6, 0) = 6

GCD(48, 18) = 6
```

Core Implementation

Bit Manipulation

```java
// Count set bits
public int countBits(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);  // Clear lowest set bit
        count++;
    }
    return count;
}

// Check if power of 2
public boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

// Single number (XOR all - pairs cancel out)
public int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
```

```
        return result;
    }
```

Math Algorithms

```
// GCD (Euclidean)
public int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

// LCM
public int lcm(int a, int b) {
    return a / gcd(a, b) * b;  // Avoid overflow
}

// Sieve of Eratosthenes
public List<Integer> sieve(int n) {
    boolean[] isPrime = new boolean[n + 1];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;

    for (int i = 2; i * i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }

    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= n; i++) {
        if (isPrime[i]) primes.add(i);
    }
    return primes;
}
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Bit operations	O(1)	O(1)
Count bits	O(k) where k = set bits	O(1)
GCD	O(log(min(a,b)))	O(1)
Sieve of Eratosthenes	O(n log log n)	O(n)

Fast exponentiation	O(log n)	O(1)

Common Problem Patterns

1. Single Number

Problem: Find element that appears once (others appear twice).

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
    return result;
}
```

2. Number of 1 Bits

Problem: Count number of set bits in integer.

```
public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);
        count++;
    }
    return count;
}
```

3. Reverse Bits

Problem: Reverse bits of a 32-bit unsigned integer.

```
public int reverseBits(int n) {
    int result = 0;
    for (int i = 0; i < 32; i++) {
        result = (result << 1) | (n & 1);
        n >>= 1;
    }
    return result;
}
```

4. Power of Two

Problem: Check if n is a power of two.

```
public boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

5. Missing Number

Problem: Find missing number in [0, n].

```
public int missingNumber(int[] nums) {
    int xor = nums.length;
    for (int i = 0; i < nums.length; i++) {
        xor ^= i ^ nums[i];
    }
    return xor;
}
```

6. Pow(x, n) - Fast Exponentiation

Problem: Implement pow(x, n).

```
public double myPow(double x, int n) {
    long N = n;
    if (N < 0) { x = 1 / x; N = -N; }
    double result = 1;
    while (N > 0) {
        if ((N & 1) == 1) result *= x;
        x *= x;
        N >>= 1;
    }
    return result;
}
```

← Previous: Trie & String Algorithms | Back to Main Guide | Next: Java Data Structures →